

Open in app ↗

≡ Medium



Level Up Coding · [Follow publication](#)

★ Member-only story

I Spent the Summer Testing 14 OCR Engines

To figure out where each excelled and where it failed

13 min read · Jul 12, 2026



Ida Silfverskiöld

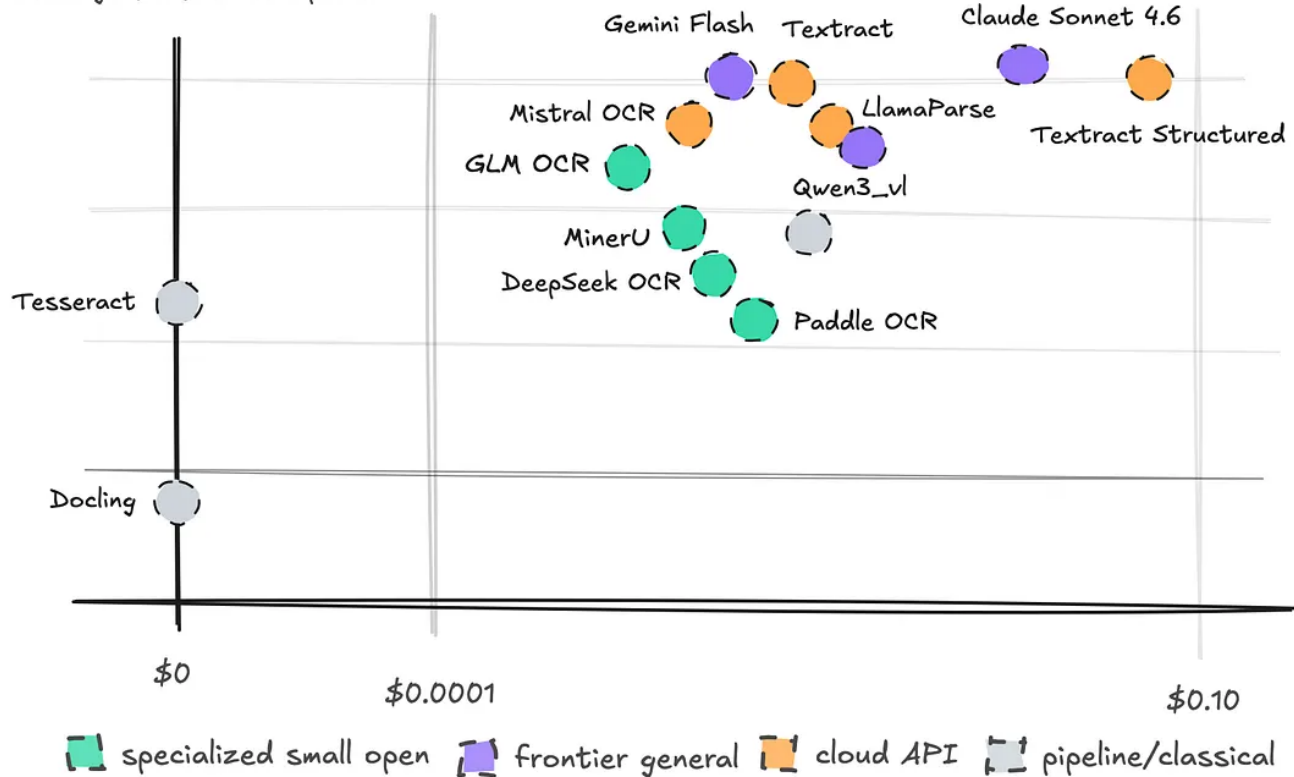
[Follow](#)

▶ Listen

📄 Share

⋮ More

Coverage (GT) vs Cost per doc

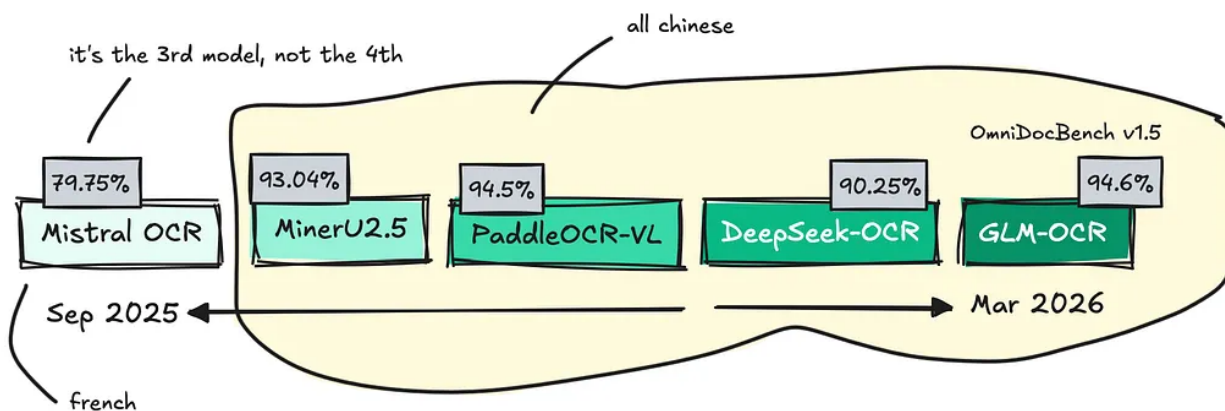


Just for fun — there is an actual graph further down | Images by author

There is a huge market for Intelligent Document Processing (IDP), and it is projected to grow to somewhere between \$20 billion and \$90 billion by the early 2030s, depending on which analyst you ask.

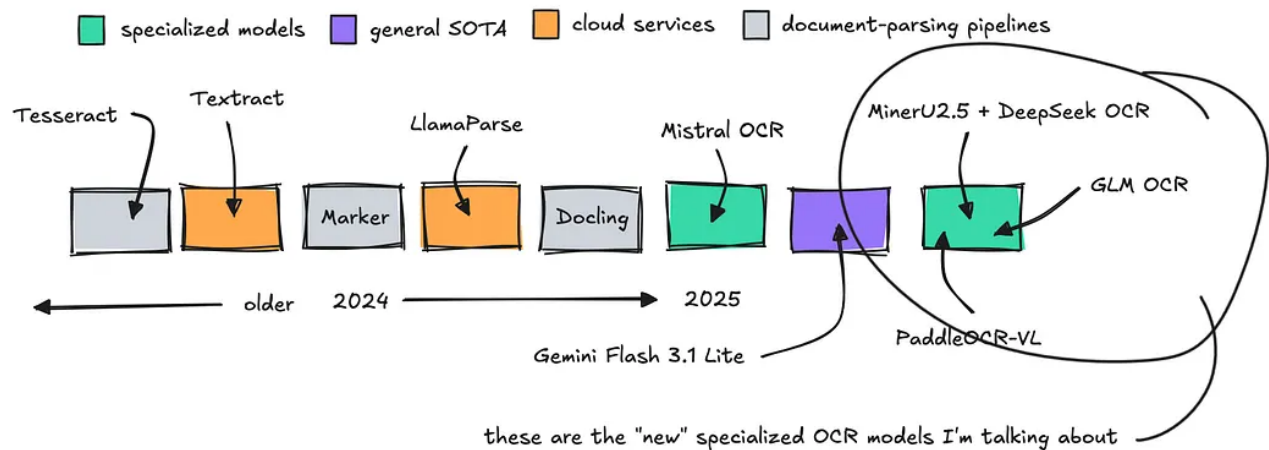
Probably driven by companies paying \$15–25 per invoice in manual handling costs.

Because I stay close to the tech world, I have watched a wave of small, specialized OCR vision models ship over the past year, mostly from Chinese organizations.



These are not the only new options, though. Document-parsing tools such as LlamaParse and Docling have also come onto the scene.

These new document-parsing models and pipelines now share the space with old-timers such as Tesseract and AWS Textract Structured. Depending on what they are being used for, the price tag for companies can currently range from \$0 to around \$65 per 1,000 processed pages.



So, I felt it would be useful to properly test 14 of these engines, old and new, on a range of messy documents, including handwritten notes, tables, legacy financial documents, scanned invoices, receipts, charts, old newspapers, and tax forms, to see how they compete.

The idea was to test how they handled two things: text recovery and the ability to preserve useful table structure.

The **main questions** I wanted answered were: **do you really need to pay \$65 per 1,000 structured pages**, or can you cut that down to a fraction? And **do the new specialized OCR models outperform the general ones?**

This article is for anyone working in AI engineering, but also for business people who need to understand how to approach this space and optimize their own setup.

TL;DR

OCR is not really a search for the best engine. It is a routing and evaluation problem, and the answer depends on your documents.

Cheap, old tools can handle clean documents. General vision models are better when documents become unpredictable. Expensive structured extraction can be necessary, but there are now cheaper options that perform just as well.

Specialized models trained specifically for OCR do well enough for the price, but

can fail sharply outside their distribution, which is impossible to see from benchmarks alone.

The key thing to carry with you is that using one engine for every document can force you to pay far more than necessary, while new technology does not always outperform older, proven tools.

Let's cover how we got to this conclusion though.

Let's run through the mechanics first

For anyone new to OCR, or anyone who needs a refresher, I'll give a brief introduction first.

Then, let's cover the test setup so you understand the engines, the types of documents I chose, and the metrics I used to decide how well each engine performed.

Explain the OCR space to me

OCR, or Optical Character Recognition, is how a machine turns a picture into machine-readable text. It is simple in principle and, for easier documents, mostly solved. It becomes harder when things get more human.

To give you a quick overview, older OCR systems found text on a page, sliced it into characters, and matched each one against a library of known shapes. Tesseract has done this since the 1980s.

Modern OCR, however, including newer versions of Tesseract, usually uses a neural network that looks at the whole page at once and outputs the document as text. So, if your document is a clean PDF or a high-quality scan in a standard font, OCR is mostly a solved problem.

It stops being solved the moment things get messier: photographed receipts, handwritten notes, weird graphs and charts, dense financial tables, or scanned tax forms and loan applications.

Companies need this done well for obvious reasons, as every downstream system acts on the output. The better OCR gets, the more paperwork becomes something a

system can reason over instead of something a human has to read by hand.

There is also the fact that if we feed AI systems badly parsed documents, everything that comes after will be difficult to trust.

This raises the question I wanted to test: can small open-source models actually do the work that expensive APIs charge for, or should we look to general vision models to handle OCR too?

The docs, the engines, and the metrics

This experiment needs to cover a few things before we move on to the results: specifically what engines we used, what docs we tested with, and how we decided who won.

For the engines, I wanted a lineup that covered all the choices I talked about, this meant: old and new, open and closed, local and cloud, specialized and general.

Tesseract became the classical choice. It runs locally and is very fast. Then I added two document-parsing pipelines: **Docling** and **Marker**. Docling is slower but runs on CPU, Marker is open-weight but needing a GPU to run fast, which shows up later in the price.

Then for the new wave of specialized open OCR models: **GLM-OCR**, **PaddleOCR-VL**, **DeepSeek-OCR**, and **MinerU 2.5** (a borderline case, really a pipeline with a VLM inside). I picked them off OpenDataLab's OmniDocBench [leaderboard](#), where they ranked first, second, fourth, and fifth.

*Note for **PaddleOCR-VL**, we ran the two-part system pipeline.*

I hosted them on Modal and served the applicable ones with vLLM, batching to speed things up. I counted the scale-up time when measuring latency later.

I also added one closed purpose-built model, **Mistral OCR**, which I'd heard good things about.

On the open side, I used **Qwen3-VL** (8B, from Alibaba), also hosted on Modal with the rest of the smaller models. *I should flag that I gave it a plain transcription prompt*

rather than the optimized serving setup it was designed for, so I may not have given it a fair shot.

On the closed side, for the general models, I picked **Gemini Flash 3.1 Lite** (currently first on the IDP Leaderboard, the western counterpart built on OmniDocBench v1.5) and **Claude Sonnet 4.6**, at sixth.

For the cloud document services: **LlamaParse** and **AWS Textract**, in both its text and structured forms. Structured Textract can do far more than I asked of it. I only tested its text accuracy across the board and its table extraction against eight of the other engines.

Let's turn to the **documents**. I picked seventeen document types that were either easy, medium, or hard. Ninety-three files in all.

Easy was the stuff OCR mostly solved years ago: clean invoices and receipts. Medium came largely from the OmniAI OCR Benchmark dataset: bank statements, medical notes, photographed receipts, shipping documents, tax forms.

Hard was chosen when things turned more difficult: charts, forms, handwritten notes, weirdly scanned financial tables, legal papers, newspapers, and old legacy reports.

Some docs were really quite difficult, such as the legacy scanned docs you see below, and this was just because I was curious if some could actually do it well.

61 4441. 100

BROWN & WILLIAMSON TOBACCO CORPORATION
CAPITAL BUDGET PROJECT

DUVIA
50914

BDATE

Capital Budget (\$000) Date Submitted

HIGH SPEED TRI-FOLD COUPON CAPABILITY/MACON BRANCH

1 ☒ Maintenance of Existing Business 5 ☐ Compliance with Outside Requirements
2 ☐ Expansion of Existing Business 6 ☐ Company Improvements & Administrative Requirements
3 ☐ New Products 7 ☐ Quality Improvement
4 ☐ Cost Reduction

This Project is ☒ Proposed
☐ Approved; Proposal No. _____

ESTIMATE COST

TOTAL	CAPITAL	EXPENSE
122	122	

PROJECT DATA

Start	1	85	Completion	11	86
No.		Yr.	No.		Yr.

PROJECT DESCRIPTION

This project provides for equipping one production module (4 GD-4350 machines) with high speed tri-fold coupon placer capability. It includes the purchase of Thiele coupon placers, hot melt adhesive applicators, and adaptation of the GD-4350 machines.

ESTIMATED SPENDING SCHEDULE

CAPITAL	EXPENSE	CAPITAL	EXPENSE
		122	

1986 SPENDING BY QUARTER

JAN.-MAR.	APR.-JUN.	JUL.-SEP.	OCT.-DEC.	TOTAL	1987	
CAPITAL	32	30	30	122	1988	
EXPENSE					1989	
					1990	

APPROVED BY

52606779

Messy legacy reports we ran through an LLM judge sourced from the [Industry Documents library](#) under fair-use license — every engine did badly according to the judge (except Gemini Flash) maybe some bias carried through there.

Some of these images came with gold ground truth and some didn't, and the ground truth I did have wasn't always consistent, some files labeled correctly, some not, which is why we should briefly cover the metrics too.

Since every engine emits different markup, the usual scoring didn't quite fit. One might pick **Precision** and **Recall** for a case like this.

***Precision** looks at how many of the OCR output's words actually match in the ground truth (GT) while **Recall** measures how many times each GT word was captured.*

Precision would punish engines that emit markdown structure the GT doesn't contain, furthermore the GT sometimes skipped labels entirely which would punish

the engine unfairly. Recall would measure the words but punish the frequency.

So, I added on a third metric called **Coverage**.

I just wanted to measure how much of the ground truth shows up somewhere in the engine's output. It isn't perfect, but it tells me whether an engine caught most of what mattered, without penalizing it for gaps that were the ground truth's fault rather than the engine's.

For the documents with no gold ground truth at all, I fell back on an LLM judge, with Gemini 3 Pro as the base model as the judge, and anyone who's used an LLM judge knows this is fickle business.

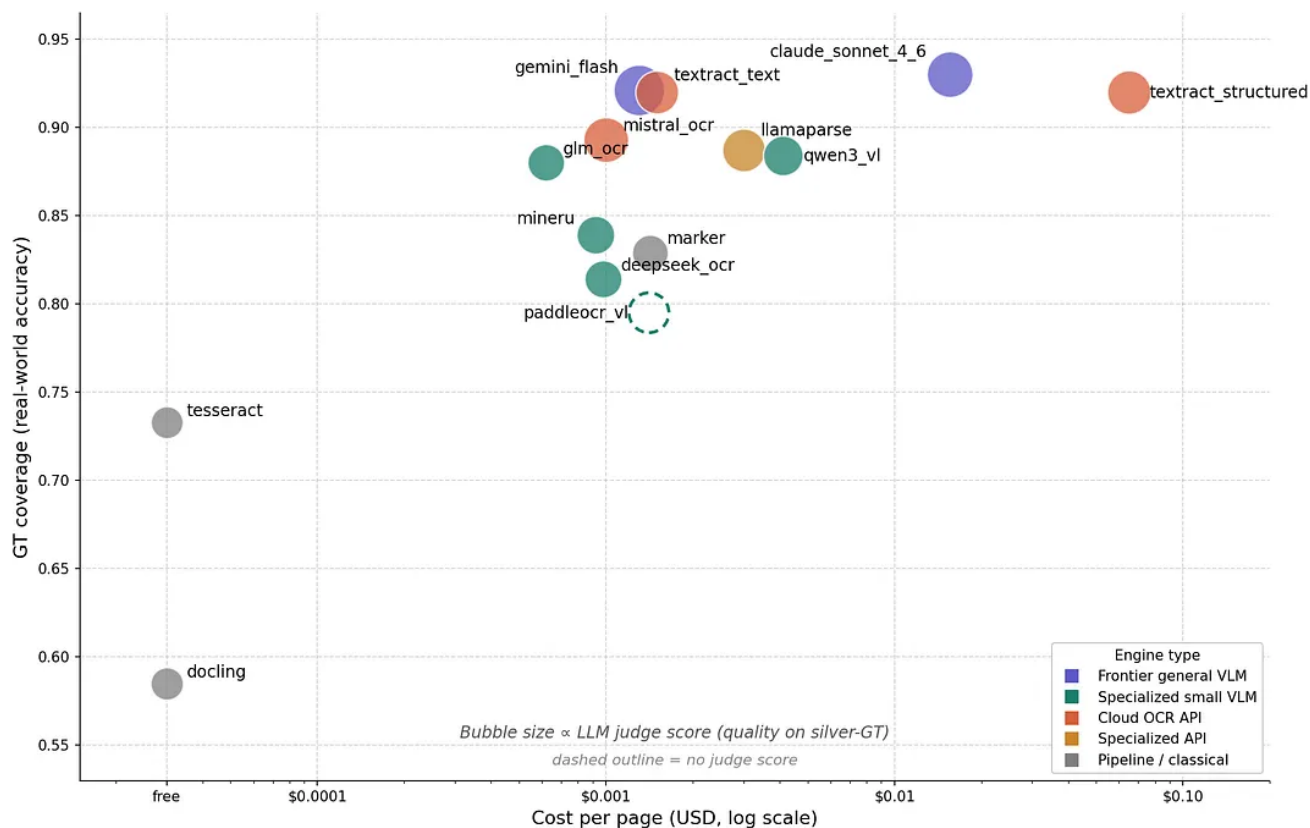
What this experiment showed

Let's briefly run through the results, look at which engines work best where, and then cover the other oddities from this test that are worth learning from.

Let's first map out the results

We mapped every document against the **Coverage** metric to build a scatter chart, and tracked latency on a separate chart. The thing a generalized chart can't tell you though is that the engines failed in different ways.

The bubble graph showed that most engines fall somewhere in the middle top, with two outliers on both sides of it.

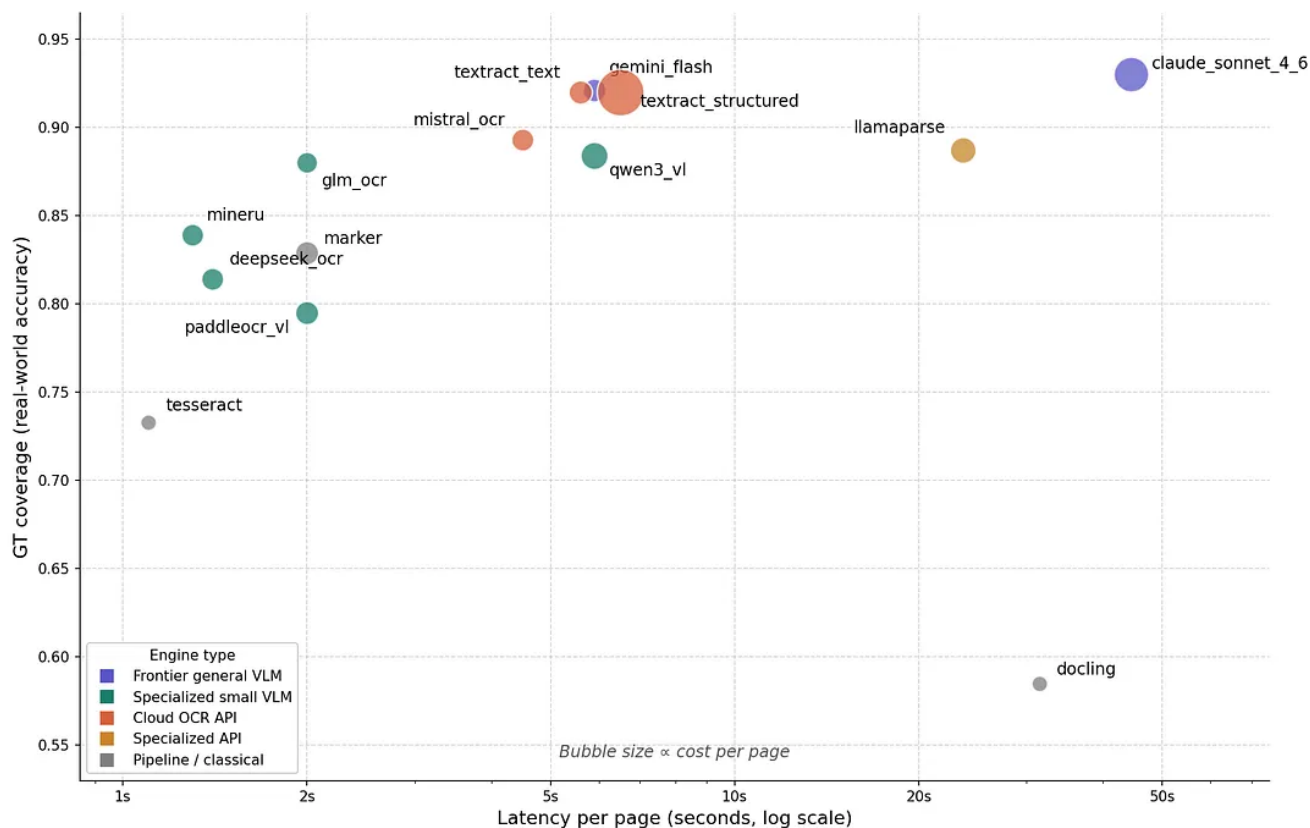


All images have been created from the result of the experiment

Gemini Flash and Textract Text did very well across the board with some edge cases. Some of the open source models like GLM and Qwen3 VL did just below the APIs but what stood out is the Coverage for GLM while being fast (it's a 0.9B model after all).

Sonnet performed the highest but also with a steeper price tag.

When we also mapped latency, some of the models turned out to be very slow, but again most wound up somewhere in the middle or to the left (if small and open source).



Tesseract, Claude Sonnet 4.6, and Docling stand out. Tesseract was incredibly fast compared to all other engines. It should be your go-to for easier documents. The specialized models with 0.9B — 3B params were quite fast when optimized for the correct GPU running on Modal.

These graphs generalise across all the documents, but I did separate the results based on the type and difficulty level.

To start with the easy docs. On invoices, every engine did well, Tesseract especially. Receipts knocked everyone down a little.

Word Precision / F1 / GT Coverage by doc type – easy tier

Engine	-- invoice --					-- receipt --				
				Fails					Fails	
	Prec↑	F1↑	Cov↑	<.5	<.7	Prec↑	F1↑	Cov↑	<.5	<.7
gemini_flash	0.594	0.733	0.961	0/6	0/6	0.843	0.876	0.901	0/6	0/6
claude_sonnet_4_6	0.592	0.732	0.961	0/6	0/6	0.840	0.873	0.899	0/6	0/6
textract_structured	0.595	0.733	0.959	0/6	0/6	0.825	0.859	0.886	0/6	0/6
textract_text	0.595	0.733	0.959	0/6	0/6	0.825	0.859	0.886	0/6	0/6
llamaparse	0.395	0.550	0.961	0/6	0/6	0.656	0.745	0.883	0/6	0/6
qwen3_vl	0.594	0.733	0.963	0/6	0/6	0.838	0.859	0.866	0/6	0/6
mistral_ocr	0.593	0.732	0.959	0/6	0/6	0.816	0.833	0.864	0/6	0/6
glm_ocr	0.595	0.728	0.942	0/6	0/6	0.851	0.866	0.874	0/6	0/6
paddleocr_vl	0.594	0.729	0.948	0/6	0/6	0.824	0.828	0.833	0/6	0/6
mineru	0.588	0.728	0.959	0/6	0/6	0.779	0.781	0.793	0/6	1/6
deepseek_ocr	0.577	0.692	0.856	0/6	1/6	0.832	0.855	0.882	0/6	0/6
marker	0.588	0.724	0.957	0/6	0/6	0.758	0.748	0.768	1/6	1/6
tesseract	0.596	0.733	0.955	0/6	0/6	0.653	0.668	0.704	0/6	3/6
docling	0.452	0.500	0.564	2/6	6/6	0.648	0.582	0.568	3/6	4/6

The one outlier was Docling, which struggled across a lot of the categories, even the easy ones.

When I looked into the Docling failures I found things like `Ifjointreturn` instead of “joint return,” and worse, strings like

`City,wrostffiel fouaveaoreignadresalcomletacesb` . DeepSeek also missed key details here like invoice number and date, which is why its number sits low.

Textract was the best engine on a lot of the medium types, along with Claude Sonnet 4.6 and Mistral OCR.

On the harder types, Gemini Flash started rising, beating Textract on forms and handwritten notes, matching it elsewhere. It did remarkably well everywhere. Tesseract and Docling failed hard on handwritten, and forms were tough for them too.

For the docs with no ground truth (newspapers, legal, reports, some scanned legacy documents) we used an LLM judge. These are genuinely hard, so it’s no surprise almost everyone failed on the reports and newspapers.

Except **Gemini Flash** that did reasonably well everywhere. Mistral OCR also did well for newspapers. Gemini Flash won everywhere with the judge, though we used Gemini Pro as the judge so take that with a grain of salt (but I did double check

myself).

Before rounding off: I also ran 8 engines against Textract Structured to see how they did on financial tables, extracting an HTML table.

I used Textract Structured's output as the ground truth for TEDS (Tree Edit Distance Similarity) and scored Claude Sonnet 4.6, LlamaParse, Mistral OCR, Gemini Flash, Marker, MinerU, DeepSeek-OCR, and Docling against it.

TABLE EXTRACTION - TEDS vs textract_structured GT (1.0 = perfect match)						
Engine	000	001	002	003	004	mean
textract_structured	1.000	1.000	1.000	1.000	1.000	1.000
mistral_ocr	0.891	0.792	0.981	0.972	0.967	0.921
llamaparse	0.841	-	-	0.972	0.912	0.909
claude_sonnet_4_6	0.891	0.792	0.902	0.972	0.974	0.907
marker	0.880	0.791	0.900	0.963	0.967	0.900
deepseek_ocr	0.858	0.814	0.900	0.917	0.953	0.888
gemini_flash	0.891	0.659	-	0.972	0.973	0.874
docling	0.832	0.712	0.768	0.925	0.927	0.833
mineru	0.889	0.396	0.790	0.972	0.971	0.804
'-' = flat text output, no table structure to extract						

Mistral OCR, and LlamaParse, and Sonnet did very well while being much cheaper. I also ran it through an LLM judge, and the winners were the same three (even before Textract Structured) though I'd want to build that test better before I fully trust it.

TABLE JUDGE SCORES - Gemini evaluates EXTRACTED TABLE vs source image										
Scale: 0-5 per dimension accuracy=cell values structure=rows/cols completeness=all rows present										
Engine	000/acc	001/acc	002/acc	003/acc	004/acc	ov↑	acc↑	str↑	cmp↑	
claude_sonnet_4_6	5.0	5.0	5.0	5.0	5.0	4.20	5.00	3.80	4.80	
llamaparse	5.0	-	-	5.0	5.0	4.00	5.00	3.67	5.00	
textract_structured	4.0	4.0	5.0	5.0	5.0	3.20	4.60	2.80	3.60	
mineru	4.0	5.0	3.0	5.0	5.0	3.00	4.40	2.60	3.60	
gemini_flash	5.0	2.0	-	5.0	5.0	3.75	4.25	3.25	4.00	
mistral_ocr	5.0	5.0	2.0	5.0	4.0	3.80	4.20	3.60	4.80	
marker	4.0	2.0	5.0	4.0	3.0	3.40	3.60	3.80	4.80	
docling	3.0	3.0	3.0	3.0	4.0	2.60	3.20	3.00	3.80	
deepseek_ocr	2.0	4.0	2.0	4.0	4.0	2.80	3.20	3.00	3.60	
'-' = no structured table extracted (flat-text engine or extraction failed)										

Now, let's talk about what it costs to scale this up, and what would make sense where.

Let's spell it out: what does best where

Let's run through what it costs to scale up with these engines, and then based on

these docs what you would chose where.

First, the cost of using these engines vary wildly, as you saw before. Sometimes it helps to see the cost not just for one document, but thousands up to a million.

Scale projection (steady-state; startup/model-load time excluded when reported)

Engine	pages/hr	\$/page	cost@1K	cost@10K	cost@100K	cost@1M	hrs@1K	hrs@10K	hrs@100K	hrs@1M
textract_structured	555	\$0.06500	\$ 65.00	\$ 650.00	\$ 6,500.00	\$ 65,000.00	1.8h	18.0h	180.1h	1800.5h
claude_sonnet_4_6	81	\$0.01555	\$ 15.55	\$ 155.47	\$ 1,554.71	\$ 15,547.06	12.4h	123.7h	1237.1h	12371.4h
qwen3_vl	607	\$0.00411	\$ 4.11	\$ 41.13	\$ 411.30	\$ 4,113.04	1.6h	16.5h	164.6h	1646.3h
llamaparse	153	\$0.00300	\$ 3.00	\$ 30.00	\$ 300.00	\$ 3,000.00	6.6h	65.6h	655.6h	6555.7h
textract_text	639	\$0.00150	\$ 1.50	\$ 15.00	\$ 150.00	\$ 1,500.00	1.6h	15.7h	156.5h	1565.1h
marker	1,760	\$0.00142	\$ 1.42	\$ 14.20	\$ 141.98	\$ 1,419.79	0.6h	5.7h	56.8h	568.3h
paddleocr_vl	1,770	\$0.00141	\$ 1.41	\$ 14.11	\$ 141.13	\$ 1,411.31	0.6h	5.6h	56.5h	564.9h
gemini_flash	610	\$0.00130	\$ 1.30	\$ 13.00	\$ 130.00	\$ 1,300.00	1.6h	16.4h	164.0h	1639.8h
mistral_ocr	796	\$0.00100	\$ 1.00	\$ 10.00	\$ 100.00	\$ 1,000.00	1.3h	12.6h	125.6h	1255.8h
deepseek_ocr	2,561	\$0.00098	\$ 0.98	\$ 9.75	\$ 97.55	\$ 975.46	0.4h	3.9h	39.0h	390.4h
mineru	2,710	\$0.00092	\$ 0.92	\$ 9.22	\$ 92.19	\$ 921.87	0.4h	3.7h	36.9h	369.0h
glm_ocr	1,786	\$0.00062	\$ 0.62	\$ 6.17	\$ 61.68	\$ 616.76	0.6h	5.6h	56.0h	559.9h
docling	114	\$0.00000	\$ 0.00	\$ 0.00	\$ 0.00	\$ 0.00	8.7h	87.5h	875.0h	8749.8h
tesseract	3,364	\$0.00000	\$ 0.00	\$ 0.00	\$ 0.00	\$ 0.00	0.3h	3.0h	29.7h	297.3h

We’re self-hosting on Modal, so these costs come from actual usage there.

We’ve calculated that the GLM OCR 0.9B model does well enough on an A10G while running qwen3_vl, mineru, deepseek_ocr, marker, and paddleocr_vl on an A100.

You can obviously run locally, but my computer wouldn’t allow it and I didn’t want to try it and this gives us numbers to work with.

For the docs we have here, Gemini Flash 3.1-Lite is a clear winner but so is GLM OCR for that price and latency. Mistral OCR did well on structured tables while staying cheap. GLM OCR did very well when we look at latency too (considering we’re self hosting).

Claude Sonnet 4.6 did well too, but it’s slow and expensive comparatively.

Docling is so very slow on my laptop. I’m sure there are ways to speed it up, but it also failed in ways that make it inherently unstable (still a small test though).

Textract is a stable choice, but structured buys you almost no additional text accuracy so if you’re paying that steep markup for structured output, make sure you actually use it. *I’m guessing it’s a pretty good business model for them.*

So, in general for this very small test: for clean, high-volume print, just use Tesseract. For general heterogeneous production, go Gemini Flash or GLM OCR. For

a cost-floor with table structure, test Mistral OCR. For high-stakes docs, route to Sonnet or a larger model.

Since everyone did well in different ways you'll have to contact me for specifics.

Let me just quickly talk about some things that stood out after doing this experiment.

What this whole experiment taught me

A handful of things surfaced from this that are worth pulling out on their own.

First, if you want to understand how a model or engine will do on your docs, the only way is to **test on those docs, you can't rely on benchmarks** to tell you. This was the number one insight this showed.

OCR usefulness depends on your own document mix, layouts, languages, scans, tables, handwriting, and failure tolerance.

Do not pay for structure if you don't need it. I wonder how many are using certain APIs or models for a reason they can't justify. Map the cost to understand what you are losing by not using the correct engine for the documents.

Lastly, the failure modes told me more than the averages.

DeepSeek OCR has chart blindness and empty outputs on some docs. While Docling has character errors, word-merging, and column misalignment all stacking together. Tesseract did fine on clean docs (as mentioned) but failed on photo/handwriting altogether outputting garbage.

Caveats to consider

Before we round up, let me cover how this test is ultimately imperfect by naming the issues in the GT, the metrics used, and the sample size.

I covered this in one of the section above, but the ground truth differs between documents depending on the dataset where they were found. In general tokenization artifacts can make correct OCR look worse than it is.

Most engines have different formats, some return plain text, some markdown, some HTML/rich markdown and it's hard to generalize across all.

We are using **Coverage**, and then also some other metrics, but these aren't perfect. Coverage won't charge the engine if it outputs too much text or the structure of it is off. Though I did find that for the engines that failed, they did so at the start or mid-way through rather than at the end.

This means it's useful for ranking but not a perfect way to score.

LLM judges are not neutral truth: I've covered this in the past, but they are biased and very prompt sensitive.

Then I just need to say that this test is interesting but not that big, the sample size is way too small to use this as a factual study. But, I don't fully trust these metrics nor the judge so it was the only way for me to be able to double check the results on my own without this turning into a year long project.

So, this test is useful for direction and getting a sense of what works, but for getting a sense of your use case, you need to run it through with your specific docs.

Lastly, latency and reproducibility is unstable. Serverless cold starts make timing noisy, and API models can silently change over time.

Like always with these articles, it takes quite a bit to do an experiment like this, but I do it because I'm genuinely curious.

What it looks like though is that OCR seems to be a routing problem, and perhaps an evaluation problem. Classify your docs and run them through several engines, then try to build a decent router and validator in your pipeline to escalate failures and then log the costs.

But we can say there are now new options for harder documents that can be just as fast and cheaper than before.

If you appreciated the article clap and give me a comment on your thoughts. You can follow my writing on my [website](#) or connect with me via [LinkedIn](#).



All datasets used in this benchmark are publicly available and sourced from HuggingFace. Licenses include MIT, CC-BY-4.0, and fair-use frameworks (UCSF Industry Documents Library) covering research, scholarship, and education. No source documents are reproduced — datasets were used solely as evaluation inputs to measure OCR engine performance.

Data Science +

Programming ✓

Software Development ✓

Ocr +

LLM +



Follow

Published in Level Up Coding

350K followers · Last published 1 day ago

Coding tutorials and news. The developer homepage [gitconnected.com](#) && [skilled.dev](#) && [levelup.dev](#)



Follow

Written by Ida Silfverskiöld

6.5K followers · 40 following

Building stuff | Connect 🤝 [github.com/ilsilfverskiold](#) & [linkedin.com/in/ida-silfverskiold](#) | [ilsilfverskiold.com](#) |
Tech intelligence 🧡 [safron.io](#)



